

# Database development with PL/SQL

## Reference reading



- [Auditing Enhancements \(Audit Policies and Unified Audit Trail\) in Oracle Database 12c Release 1 \(12.1\)](#)

## Lecture 10 – Advanced Auditing in PLSQL

# Database development with PL/SQL INSY 8311



ADVENTIST UNIVERSITY  
OF CENTRAL AFRICA

## Instructor:

- Eric Maniraguha | [ericmaniraguha2024@gmail.com](mailto:ericmaniraguha2024@gmail.com) | [LinkedIn Profile](#)



6h00 pm – 8h50 pm

- Monday A -G209
- Tuesday B-G209
- Wednesday E-G209
- Thursday F-G308

**November 2024**

# The objectives of Auditing in Databases | Key Components of Database Auditing

Auditing in databases refers to monitoring, recording, and analyzing user activities within the database. This includes tracking SQL operations, user access, and other actions that involve data. It serves to ensure accountability, compliance, and the detection of unauthorized or suspicious activities.

## Activities Monitored:

- Actions on specific schemas, tables, or rows.
- Successful or failed database access attempts.
- Privileged user actions (e.g., SYSDBA operations).

## Configuration:

- Enabling auditing through initialization parameters (e.g., AUDIT\_TRAIL).
- Setting up audit policies to target specific activities or data.

## Audit Record Locations:

- Database audit trail (e.g., SYS.AUD\$).
- Operating system files (e.g., UNIX syslog or Windows Event Viewer).

## Auditing Types:

- Standard Auditing:** Focuses on SQL statements, privileges, and schema objects.
- Fine-Grained Auditing:** Offers detailed auditing at the data level (e.g., actions based on specific conditions like IP address).

## Types of Users in Auditing:

- Database Users:** Traditional users who directly interact with the Oracle database.
- Nondatabase Users:** Users of applications that interact with the database. These users are identified in the database using the CLIENT\_IDENTIFIER attribute.
- Auditing nondatabase users is more complex as they interact indirectly through the application.

# Oracle Audit Vault

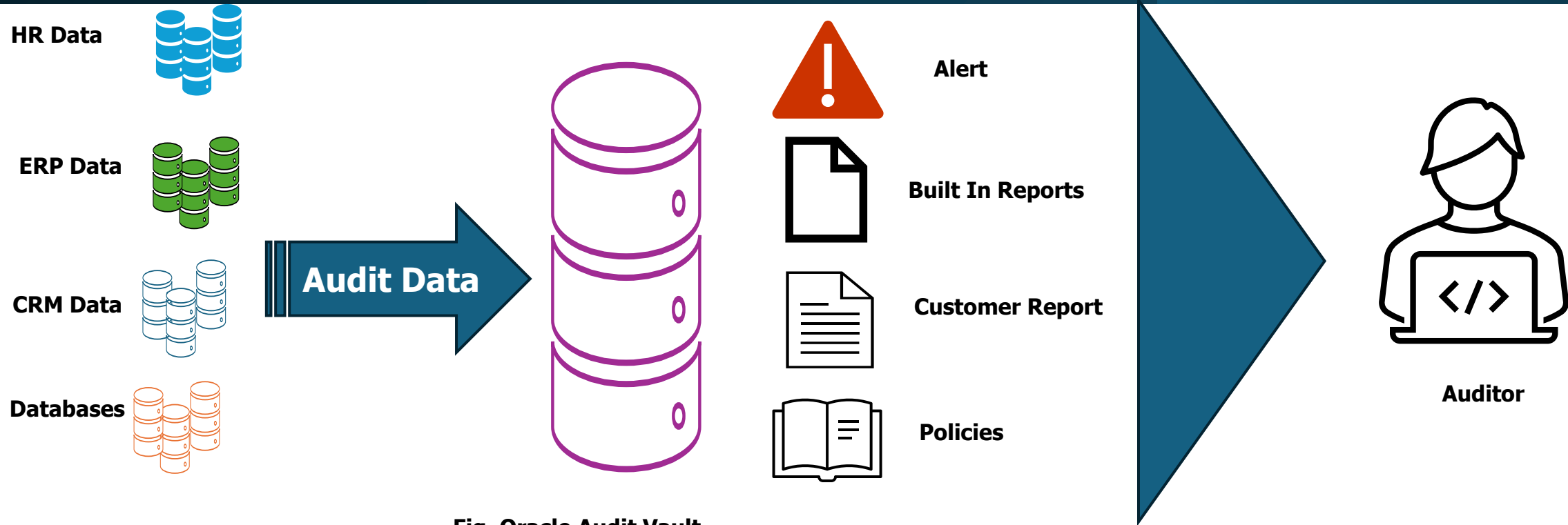


Fig. Oracle Audit Vault

## Purpose:

This diagram highlights how Oracle Audit Vault centralizes auditing for various data systems, enabling streamlined monitoring, compliance enforcement, and auditing for organizations. It shows the end-to-end process of collecting, storing, analyzing, and reporting data to assist auditors in decision-making.

# Oracle Audit Vault – Diagram Key Elements and Workflow

## Data Sources (Left side of the diagram) - The monitored systems include:

- **HR Data:** Employee-related databases.
  - **ERP Data:** Enterprise resource planning systems.
  - **CRM Data:** Customer relationship management systems.
  - **Databases:** General-purpose databases storing organizational data.
- These systems generate **audit data**, which includes logs of user activity, data access, and any changes made.

## Audit Data Collection and Storage (Middle section):

- **Audit Data:** The logs and audit trails from the data sources are collected and transported to a central **Audit Data Repository** (the pink database icon).
- This repository serves as the backbone for securely storing all audit logs in a structured format for further analysis.

## 3. Processing and Insights (Middle elements on the right of the repository): - The stored audit data is processed to generate actionable insights:

- **Alerts:** Notifications triggered by suspicious activities, such as unauthorized access or policy violations.
- **Built-In Reports:** Predefined, standard reports for common auditing tasks (e.g., user activity logs, failed login attempts).
- **Customer Reports:** Customized reports tailored to specific compliance or business requirements.
- **Policies:** Rules and monitoring criteria configured to enforce security and regulatory compliance.

## 4. Output for Auditors (Right side of the diagram):

- The processed insights (alerts, reports, and policies) are made available to the **Auditors**.
- Auditors use this information to:
  - Monitor system activities.
  - Investigate any violations or unusual behavior.
  - Ensure compliance with security standards and regulatory requirements.

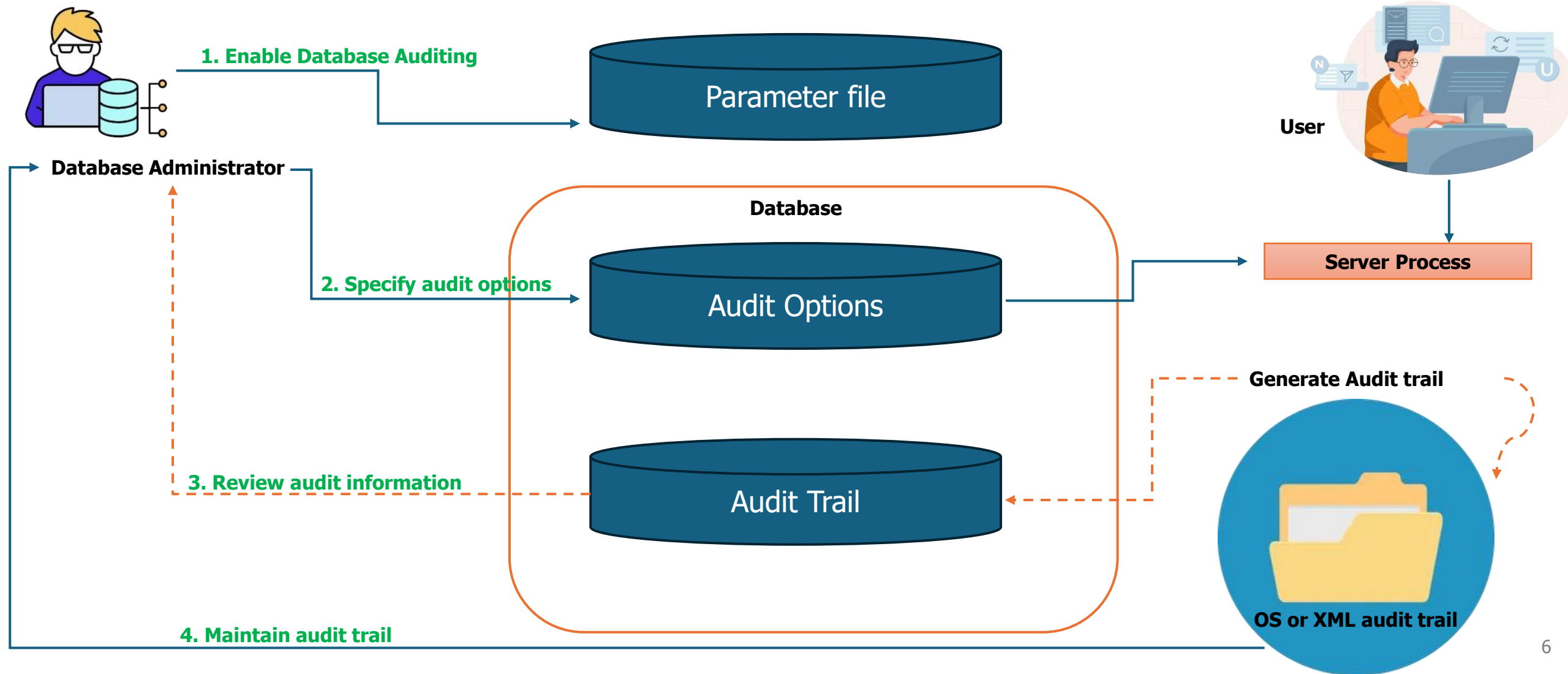
## Purpose of the System:

Oracle Audit Vault centralizes auditing and monitoring across multiple systems, providing organizations with:

- A secure and consolidated way to manage audit logs.
- The ability to detect suspicious activities in real time.
- Tools for creating reports to comply with internal policies and external regulations.

This end-to-end process ensures effective oversight of organizational data, reducing security risks and improving compliance.

# Standard Database Auditing



# The Purpose and Scope of Auditing: Why and Who We Audit

## Importance of Auditing

- Acts as a deterrent against malicious activities.
- Forms the foundation of security initiatives.
- Helps identify and track access to sensitive objects.
- Supports investigations and compliance requirements.
- Warns of suspicious activities.
- Provides insights into the current state of access and activities to define effective access control policies.

## Audit Trail

- **Key Information Captured**
- **Who** performed the action.
- **Where** it occurred (Schema/Object).
- **When** it occurred.
- **How** it was performed (SQL statements executed).

Crucial for understanding user activities and ensuring accountability.

# Key Steps in the Auditing Process: A Structured Approach

## 1. Planning the Audit

- **Define Objectives:** Clarify what the audit aims to achieve (e.g., compliance, efficiency, fraud detection).
- **Understand the Scope:** Identify the areas or processes to be audited.
- **Gather Information:** Review relevant documents, policies, and regulations.
- **Develop an Audit Plan:** Outline timelines, resources, and team assignments.

## 2. Preparation

- **Notify Stakeholders:** Inform relevant parties about the audit schedule.
- **Identify Key Risks:** Determine potential issues or areas of concern.
- **Assign Roles:** Allocate tasks to audit team members.

## 3. Execution

- **Data Collection:** Gather evidence through document reviews, interviews, and observations.
- **Testing and Verification:** Test controls, processes, and systems for compliance and accuracy.
- **Analysis:** Compare findings against benchmarks, policies, and standards.

## 4. Evaluation

- **Summarize Findings:** Consolidate evidence and assess its significance.
- **Identify Gaps:** Highlight deviations, risks, or inefficiencies.
- **Determine Impact:** Evaluate the severity and potential consequences of issues found.

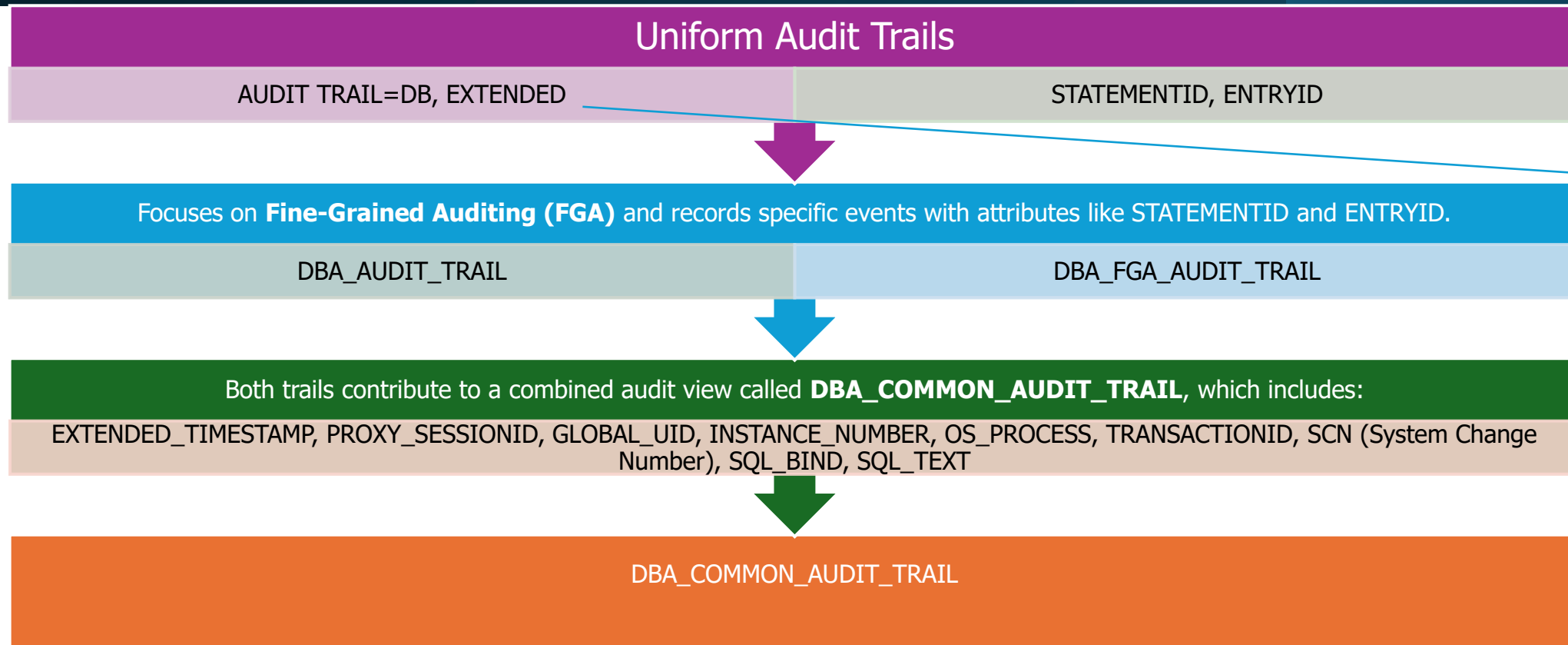
## 5. Reporting

- **Draft the Report:** Include findings, conclusions, and actionable recommendations.
- **Discuss with Stakeholders:** Present the report to the management team or relevant parties for feedback.
- **Finalize Report:** Incorporate feedback and finalize documentation.

## 6. Follow-Up

- **Monitor Implementation:** Ensure corrective actions are taken as recommended.
- **Reassess:** Conduct follow-up audits or reviews to verify resolution.
- **Continuous Improvement:** Use lessons learned to improve future audits.

# Uniform Audit Trails



## Monitoring:

- Use **DBA\_AUDIT\_TRAIL** and **DBA\_FGA\_AUDIT\_TRAIL** to review detailed logs.
  - Consolidate and analyze using **DBA\_COMMON\_AUDIT\_TRAIL** for a unified view.
- This setup allows for comprehensive monitoring, ensuring accountability and enabling root-cause analysis for database events.

Tracks detailed database activity and stores it in two audit trails

# Uniform Audit Trails Framework

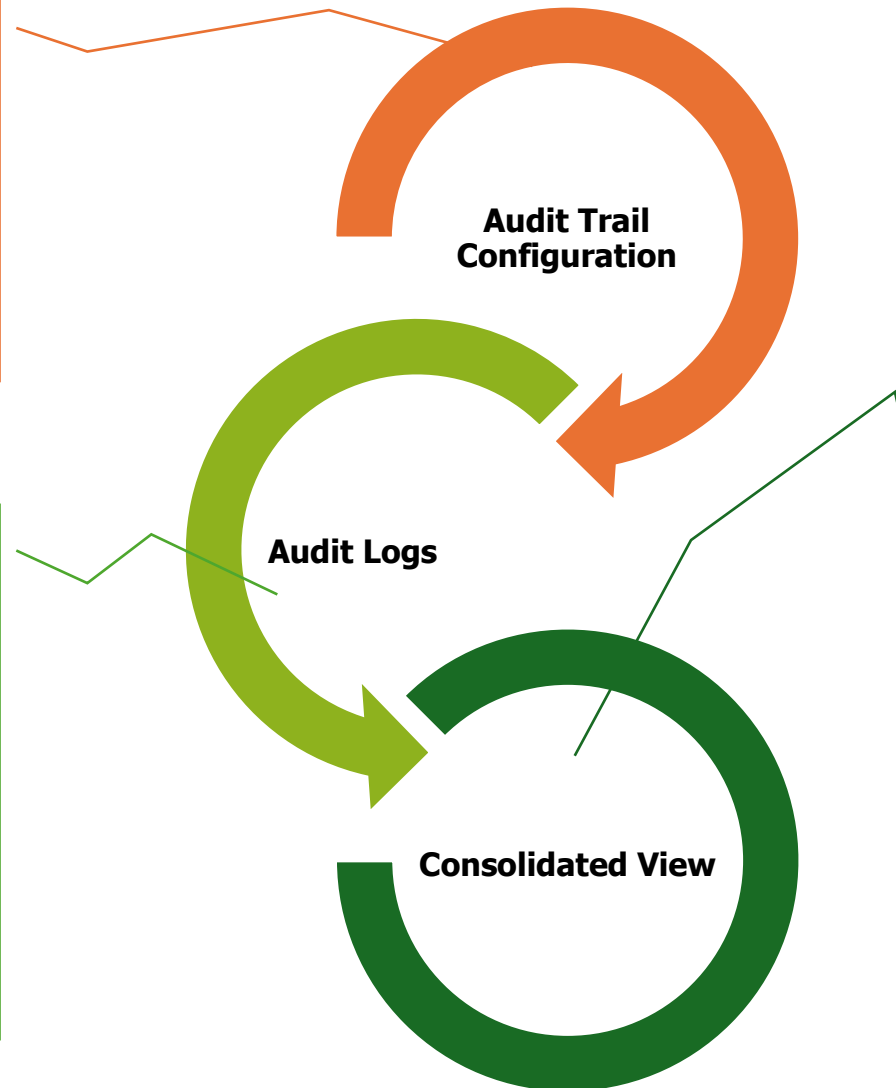


## 1. Audit Trail Configuration:

- Set `AUDIT_TRAIL=DB, EXTENDED` to enable:
  - Comprehensive auditing of SQL statements and database events.
  - Detailed logging of extended information.

## 2. Audit Logs:

- **DBA\_AUDIT\_TRAIL:**
  - Captures general database audit events (e.g., user actions, logins).
- **DBA\_FGA\_AUDIT\_TRAIL:**
  - Logs **Fine-Grained Auditing (FGA)** activities, including:
    - **StatementID:** A unique identifier for each SQL statement.
    - **EntryID:** A sequential record number for precise tracking.



## 3. Consolidated View

**DBA\_COMMON\_AUDIT\_TRAIL** combines data from both audit trails and includes extended information:

- **EXTENDED\_TIMESTAMP:** Accurate time of the event.
- **PROXY\_SESSIONID:** Tracks sessions with proxy users.
- **GLOBAL\_UID:** Unique global user identifier.
- **INSTANCE\_NUMBER:** Database instance where the event occurred.
- **OS\_PROCESS:** Operating system process ID for the session.
- **TRANSACTIONID:** Identifies related transactions.
- **SCN (System Change Number):** Database state identifier.
- **SQL\_BIND:** Details of bind variables in the statement.
- **SQL\_TEXT:** Full SQL query text executed.

# Default Auditing

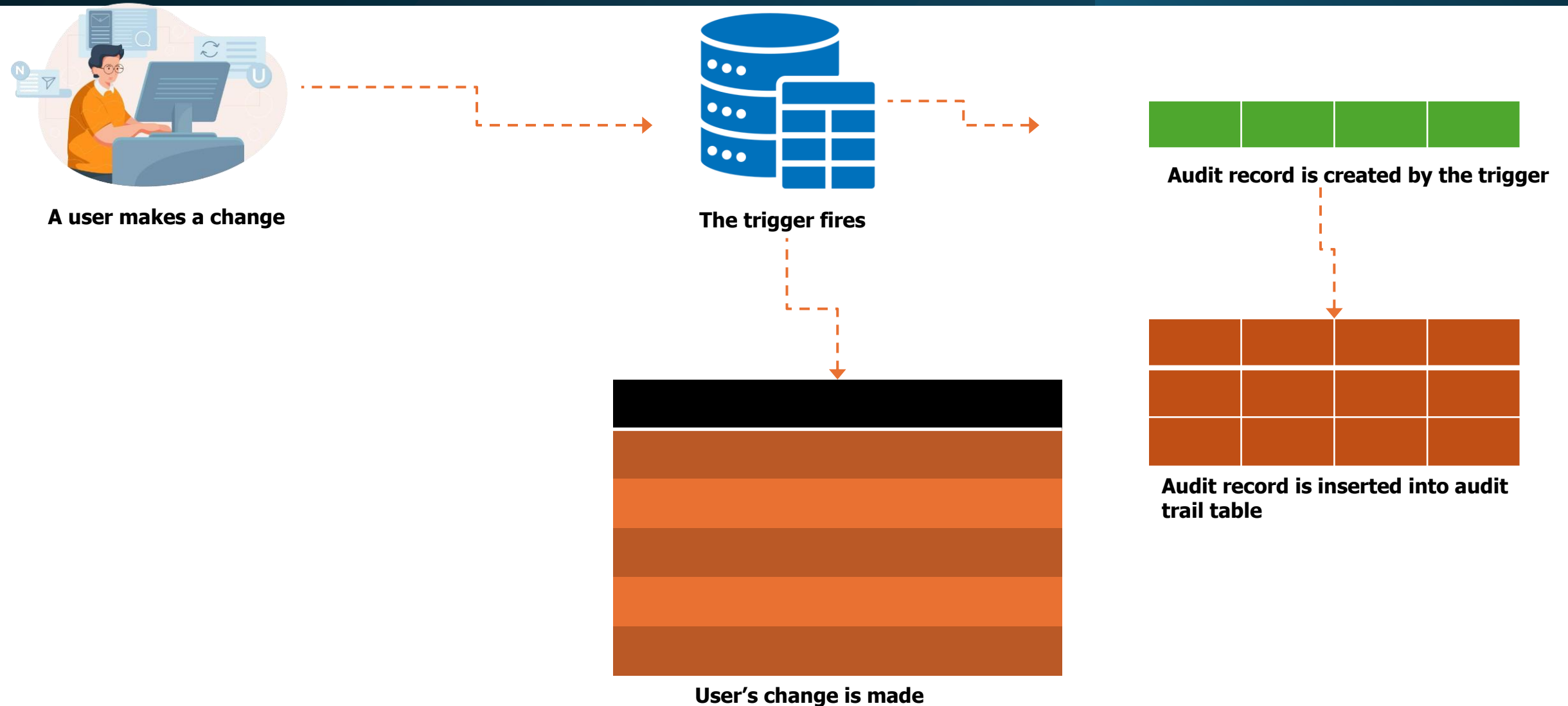
| Privileges Audited by Default                                                                                                                                       | Privileges Audited by Default                                                                                                                                                                         | Privileges Audited by Default                                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| ALTER ANY PROCEDURE<br>SYSTEM AUDIT BY ACCESS<br>ALTER ANY TABLE<br>ROLE BY ACCESS<br>ALTER DATABASE<br>ALTER PROFILE<br>ALTER SYSTEM<br>ALTER USER<br>AUDIT SYSTEM | CREATE ANY JOB<br>CREATE ANY LIBRARY<br>CREATE ANY PROCEDURE<br>CREATE ANY TABLE<br>CREATE EXTERNAL JOB<br>CREATE PUBLIC DATABASE LINK<br>CREATE SESSION<br>CREATE USER<br>GRANT ANY OBJECT PRIVILEGE | GRANT ANY PRIVILEGE<br>GRANT ANY ROLE<br>DROP ANY PROCEDURE<br>DROP ANY TABLE<br>DROP PROFILE<br>DROP USER<br>EXEMPT ACCESS POLICY |
|                                                                                                                                                                     | <b>Statement Audited by Default</b>                                                                                                                                                                   |                                                                                                                                    |
|                                                                                                                                                                     | SYSTEM AUDIT BY ACCESS<br>ROLE BY ACCESS                                                                                                                                                              |                                                                                                                                    |

**Insight:** Default auditing ensures that sensitive operations, such as granting roles, altering system configurations, or creating/dropping tables and procedures, are monitored. This is crucial for identifying unauthorized or unintended changes.

# Specifying Audit Options

| Audit Type                        | Command                                               | Description                                                                          | Disable Command                                  |
|-----------------------------------|-------------------------------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------|
| <b>SQL Statement Auditing</b>     | <i>AUDIT TABLE;</i>                                   | Audits all TABLE statements executed in the database.                                | NOAUDIT TABLE;                                   |
|                                   | <i>AUDIT LOCK;</i>                                    | Audits all LOCK statements executed in the database.                                 | NOAUDIT LOCK;                                    |
| <b>System Privileges Auditing</b> | <i>AUDIT SELECT ANY TABLE, CREATE ANY TRIGGER;</i>    | Audits any session performing SELECT ANY TABLE or CREATE ANY TRIGGER.                | NOAUDIT SELECT ANY TABLE, CREATE ANY TRIGGER;    |
|                                   | <i>AUDIT SELECT ANY TABLE BY HR BY SESSION;</i>       | Audits SELECT ANY TABLE actions by the HR user, grouped by session.                  | NOAUDIT SELECT ANY TABLE BY HR BY SESSION;       |
| <b>Object Auditing</b>            | <i>AUDIT ALL ON HR.EMPLOYEES;</i>                     | Audits all operations on the HR.EMPLOYEES table.                                     | NOAUDIT ALL ON HR.EMPLOYEES;                     |
|                                   | <i>AUDIT UPDATE, DELETE ON HR.EMPLOYEE BY ACCESS;</i> | Audits UPDATE and DELETE operations on the HR.EMPLOYEE table, recording each access. | NOAUDIT UPDATE, DELETE ON HR.EMPLOYEE BY ACCESS; |
|                                   | <i>AUDIT RENAME ON HR.EMPLOYEE;</i>                   | Audits all RENAME operations on the HR.EMPLOYEE table.                               | NOAUDIT RENAME ON HR.EMPLOYEE;                   |

# Value based Auditing



# Fine Grained Auditing (FGA)

Fine-Grained Auditing (FGA) allows you to audit specific actions at a detailed level by applying policies to table objects. It is used to monitor and log access based on conditions or specific rows in a table.

## Key Features of FGA:

**Granularity:** Audits specific rows or conditions in a table, not just general access.

**Policies:** Auditing is driven by policies that define the conditions under which auditing should occur.

**Custom Conditions:** You can specify predicates (conditions) that determine when an audit is triggered.

**Custom Handlers:** Optionally, an audit event handler can execute PL/SQL code when an audit condition is met.

**Policy: AUDIT\_EMPS\_SALARY**

```
SELECT name, salary FROM employees WHERE department_id = 10;
```

The diagram illustrates the application of Fine-Grained Auditing (FGA). On the left, a blue box labeled 'Policy: AUDIT\_EMPS\_SALARY' and an orange box containing the SQL query 'SELECT name, salary FROM employees WHERE department\_id = 10;' both have arrows pointing to a grid representing the 'Employees' table. The grid is 5 rows by 8 columns. The top row is highlighted in purple, representing the specific rows audited by the policy. The other rows are light purple.

|  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |

**Employees**

# Fine Grained Auditing (FGA)

Fine-Grained Auditing (FGA) allows you to audit specific actions at a detailed level by applying policies to table objects. It is used to monitor and log access based on conditions or specific rows in a table.

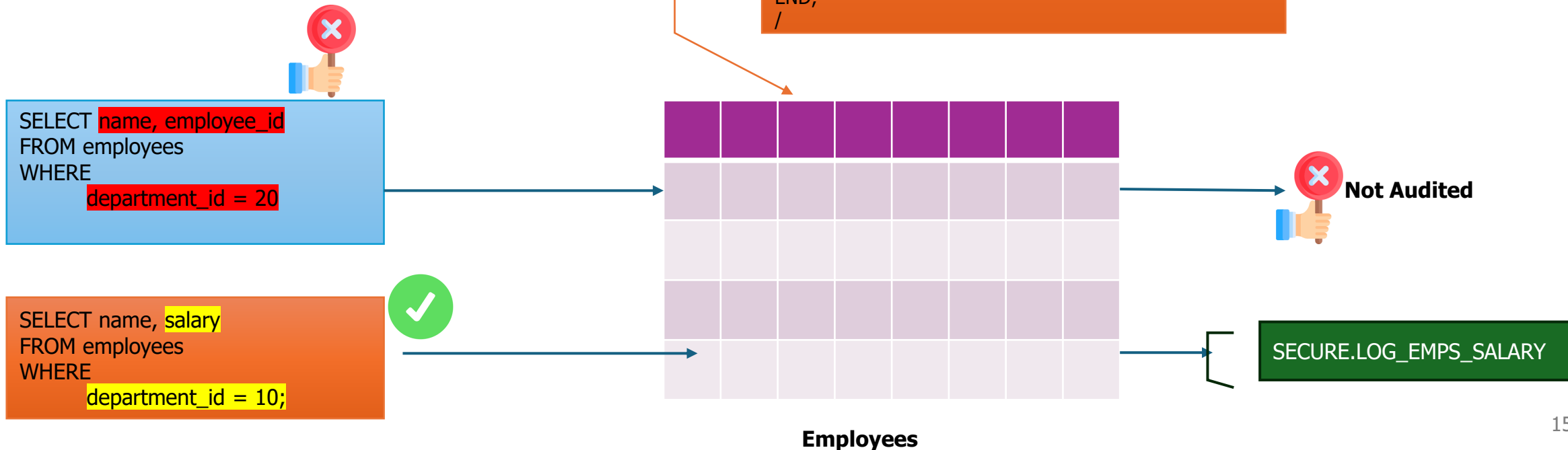
## Define:

- Audit criteria
- Audit action

## Is created with:

- DBMS\_FGA.ADD\_POLICY

```
BEGIN
DBMS_FGA.ADD_POLICY(
  object_schema => 'HR',
  object_name   => 'EMPLOYEES',
  policy_name   => 'SALARY_AUDIT',
  audit_condition => DEPARTMENT_ID = 10',
  audit_column  => 'SALARY, COMMISSION_PCT',
  handler_schema => 'SECURE',
  handler_module => 'log_emps_salary',
  enable        => TRUE
  statement_types => 'SELECT, UPDATE'
);
END;
/
```



# Audited DML Statement: Consideration

## Fine-Grained Auditing (FGA) Overview

**Definition:** FGA audits database operations based on specific conditions and referenced columns, providing targeted monitoring for sensitive data.

### Audited Actions

Operations like SELECT, INSERT, and UPDATE are audited if conditions are met or specific columns are referenced.

DELETE statements are **always audited**, regardless of conditions or columns.

## MERGE Statement Auditing

**Behavior:** The MERGE statement triggers audits for its underlying INSERT, UPDATE, and DELETE operations as independent actions.

**Implications:**

Each operation within a MERGE is checked against FGA policies.

Logs must capture operation type (e.g., INSERT or UPDATE) and relevant details like affected rows or columns.

## Key Considerations



**Policy Setup:** Ensure FGA policies cover all relevant operations (INSERT, UPDATE, DELETE) triggered by MERGE.

**Performance:** Target sensitive columns and conditions to minimize overhead.

**Audit Clarity:** Maintain detailed logs distinguishing the operation type and the affected data.

This ensures robust, compliant, and efficient auditing of both direct and composite operations like MERGE.

UPDATE hr.employees SET salary = 1000 WHERE commission\_pct = .2;

UPDATE hr.employees SET salary = 1000 WHERE employee\_id = 200;



Not audited because none of the records involved are for department 10



# Oracle Fine-Grained Auditing (FGA) Guidelines



## Audit Conditions and Columns:

**Audit All Rows:** Use a **null audit condition** (no condition specified).

**Audit All Columns:** Use a **null audit column** to apply the policy to all columns in the table.

## Policy Creation:

**Unique Policy Names:** Each FGA policy must have a unique name within the schema.

**Pre-existing Tables/Views:** The table or view to be audited must exist before creating the policy.

## Error Handling:

**Invalid Syntax:** If the audit condition has invalid syntax, accessing the audited object will raise an ORA-28112 error.

**Nonexistent Columns:** No rows are audited if the specified column in the policy does not exist in the table.

**Missing Event Handlers:** If the specified event handler (e.g., a PL/SQL procedure) does not exist, no error occurs, and the audit record is still generated.

## Best Practices:

Ensure audit conditions and columns are carefully tested to avoid unintended errors or gaps in auditing.

Regularly validate the existence and functionality of event handlers tied to FGA policies.

**By following these guidelines, you can implement robust and error-free auditing policies in Oracle Database.**

# Maintaining the Audit Trail in Oracle 12c Database (FGA/Standard Auditing)

## Best Practices for Maintaining the Audit Trail in Oracle 12c Database (FGA/Standard Auditing):

To ensure the integrity and reliability of your audit trail in Oracle Database, follow these best-practice guidelines.

### Review and Archive Old Records:

Regularly review audit records to identify any suspicious activities or patterns.

Archive historical audit data to a secure location for long-term storage, ensuring compliance with regulatory requirements.

### Prevent Storage Problems:

Monitor the available space in the tablespaces used for audit data to avoid storage issues.

Implement partitioning or purging mechanisms to manage the volume of audit logs effectively.

Use Oracle's DBMS\_AUDIT\_MGMT package to automate audit trail cleanup and management tasks.

### Avoid Loss of Records:

Enable appropriate logging mechanisms to capture all relevant audit activities, such as Fine-Grained Auditing (FGA) for detailed monitoring.

Regularly back up the audit trail to prevent data loss due to hardware failures or unexpected events.

Configure alerts for critical thresholds to proactively address storage or system issues that could impact audit logging.

By adhering to these practices, you can maintain a reliable and secure audit trail that supports both security and compliance needs within your Oracle Database environment.

# 1. Configuring the Audit Trail (1/2)

```
C:\Users\Eric>sqlplus sys/admin@localhost:1521/ORACLE21CDB AS SYSDBA
```

```
SQL*Plus: Release 21.0.0.0.0 - Production on Sun Nov 17 14:08:00 2024  
Version 21.3.0.0.0
```

```
Copyright (c) 1982, 2021, Oracle. All rights reserved.
```

```
Connected to:  
Oracle Database 21c Enterprise Edition Release 21.0.0.0.0 - Production  
Version 21.3.0.0.0
```

```
SQL>  
SQL>  
SQL>  
SQL> SHOW PDBS;
```

| CON_ID | CON_NAME       | OPEN MODE  | RESTRICTED |
|--------|----------------|------------|------------|
| 2      | PDB\$SEED      | READ ONLY  | NO         |
| 3      | ORACLE21DBCPDB | MOUNTED    |            |
| 5      | PLSQL2024      | READ WRITE | NO         |

```
SQL>  
SQL>  
SQL> show con_name;
```

```
CON_NAME
```

```
-----  
CDB$ROOT
```

-- Connect to the database

SQL> sqlplus sys as SYSDBA or

C:\Users\Eric> sqlplus sys/admin@localhost:1521/ORACLE21CDB AS SYSDBA

-- Check all pluggable database

SQL> SHOW PDBS;

-- Check all pluggable database

SQL> SHOW CON\_NAME;

# 1. Configuring the Audit Trail (2/2)

```
SQL> ALTER SESSION SET CONTAINER = CDB$ROOT;
```

```
Session altered.
```

```
SQL>
```

```
SQL> show con_name;
```

```
CON_NAME
```

```
-----  
CDB$ROOT
```

```
SQL>
```

```
SQL>
```

```
SQL> ALTER SYSTEM SET AUDIT_TRAIL = DB, EXTENDED SCOPE = SPFILE;
```

```
System altered.
```

```
SQL>
```

```
SQL>
```

```
SQL> SHUTDOWN IMMEDIATE;
```

```
Database closed.
```

```
Database dismounted.
```

```
ORACLE instance shut down.
```

```
ERROR:
```

```
ORA-12514: TNS:listener does not currently know of service requested in connect  
descriptor
```

```
Warning: You are no longer connected to ORACLE.
```

```
SQL> startup;
```

```
SP2-0640: Not connected
```

```
SQL>
```

```
SQL>
```

-- Connect to the database

```
SQL> ALTER SESSION SET CONTAINER = CDB$ROOT;
```

-- Check database in connection

```
SQL> SHOW CON_NAME;
```

## Audit trail can be set to:

- **DB:** Stores audit records in the database tables.
- **DB, EXTENDED:** Same as DB but includes SQL text.
- **OS:** Writes audit records to the operating system.
- **XML:** Stores records in an XML file.
- **NONE:** Disables auditing

--Enable auditing at the CDB\$ROOT level:

```
SQL> ALTER SYSTEM SET AUDIT_TRAIL = DB, EXTENDED SCOPE  
= SPFILE;
```

--This sets up auditing for the entire container database (CDB).

--Shut down and restart the database to apply the changes:

```
SQL> SHUTDOWN IMMEDIATE;
```

```
SQL> STARTUP;
```

# 2. Understand Where Audit Data is Stored

**AUD\$ table:** The audit records are stored in the **SYS.AUD\$** table within the database.

**DBA\_AUDIT\_TRAIL view:** Oracle provides the **DBA\_AUDIT\_TRAIL** view, which you can query to access audit records in a user-friendly format.

**Operating System or XML files:** If you set up auditing to log data externally (e.g., **AUDIT\_TRAIL = OS** or **AUDIT\_TRAIL = XML**), audit files will be generated in the specified directory.

```
/*
2. Query Audit Data
Use the following SQL queries to interact with the audit trail:
Check All Audit Records:
*/
SELECT * FROM DBA_AUDIT_TRAIL;
/*
This will show all audit records logged in the database.
Filter Audit Records by User:
*/
SELECT * FROM DBA_AUDIT_TRAIL WHERE USERNAME = 'USERNAME';
/*
Filter by Action (e.g., Login)
*/
SELECT * FROM DBA_AUDIT_TRAIL WHERE ACTION_NAME = 'LOGON';
/*
Check Audit Policies:
*/
SELECT * FROM DBA_OBJ_AUDIT_OPTS;
```

# 3. Monitory Audit log

---

For real-time monitoring of audit logs, consider using Oracle tools such as:

---

**Oracle Enterprise Manager (OEM):** Provides a graphical interface for monitoring and managing audit logs.

---

**PL/SQL scripts or shell scripts:** Create automated scripts to extract and analyze audit data periodically.

# 4. Manage Audit Logs

For real-time monitoring of audit logs, consider using Oracle tools such as:

```
/*
Archive Audit Records: Export the audit logs periodically:
*/

expdp system/your_password schemas=SYS tables=AUD$ directory=DATA_PUMP_DIR
dumpfile=audit_logs.dmp logfile=audit_logs.log;

/*
Purge Audit Records: Use the DBMS_AUDIT_MGMT package to clean up old audit data:
*/

BEGIN
  DBMS_AUDIT_MGMT.INIT_CLEANUP(
    AUDIT_TRAIL_TYPE => DBMS_AUDIT_MGMT.AUDIT_TRAIL_AUD_STD,
    USE_LAST_ARCH_TIMESTAMP => TRUE
  );
  DBMS_AUDIT_MGMT.CLEAN_AUDIT_TRAIL(
    AUDIT_TRAIL_TYPE => DBMS_AUDIT_MGMT.AUDIT_TRAIL_AUD_STD,
    USE_LAST_ARCH_TIMESTAMP => TRUE
  );
END;
/
```

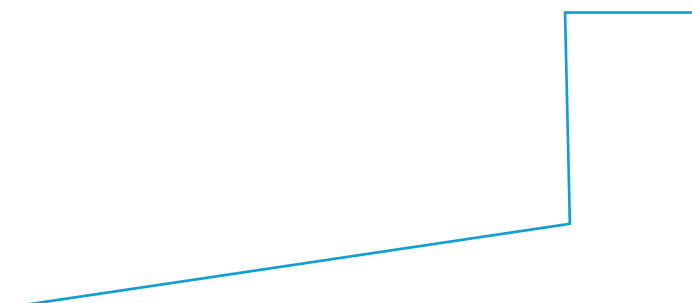
# 5. Configure Specific Audit Policies

## 6. Use Unified Auditing (Optional)



**To audit specific actions or objects**

```
/*  
Audit SQL Statements:  
*/  
AUDIT SELECT TABLE BY USERNAME;  
  
/*  
Audit Object Access:  
*/  
AUDIT INSERT, UPDATE, DELETE ON SCHEMA.TABLE;
```



**Use Unified Auditing (Optional)**

```
/*  
If your database is configured for Unified Auditing, use UNIFIED_AUDIT_TRAIL  
*/  
SELECT * FROM UNIFIED_AUDIT_TRAIL WHERE EVENT_TIMESTAMP > SYSDATE - 1;  
  
/*  
To enable specific audit policies:  
*/  
CREATE AUDIT POLICY login_audit_policy  
  ACTIONS LOGON, LOGOFF;  
AUDIT POLICY login_audit_policy;
```

# Access Audit Vault (Optional) & Security Best Practices

**7. Access Audit Vault (Optional):** If you have configured Oracle Audit Vault, use the Audit Vault interface to:



☐ View, manage, and report on audit logs.

☐ Set up alerts for suspicious activities.

## 8. Security Best Practices



☐ Regularly review audit logs for suspicious activities.

☐ Limit access to audit data to authorized users only.

☐ Rotate and archive logs to comply with regulatory requirements.

☐ By leveraging these approaches, you can effectively interact with and manage your audit trail.

# Keep Learning!